

Formalizing Formal Group Laws

Wenrong Zou

Mathematical Institute of the University of Bonn

02.07.2026

Table of contents

- 1 Motivation
- 2 Definition
- 3 Algebraic structure
- 4 Homomorphisms and isomorphisms
- 5 Conclusion

Motivation

A *formal group law* is a formal power series $F(X, Y)$ that behaves like an additive group, up to formal substitution.

Motivation

A *formal group law* is a formal power series $F(X, Y)$ that behaves like an additive group, up to formal substitution.

It appears in many areas:

- **Algebraic number theory:** Lubin–Tate Theory uses formal group laws to construct the maximal abelian extension of a local field, and gives an explicit formula for the local reciprocity map.

Motivation

A *formal group law* is a formal power series $F(X, Y)$ that behaves like an additive group, up to formal substitution.

It appears in many areas:

- **Algebraic number theory:** Lubin–Tate Theory uses formal group laws to construct the maximal abelian extension of a local field, and gives an explicit formula for the local reciprocity map.
- **Arithmetic geometry:** Every elliptic curve has an associated formal group law (the completion at the identity). Its height distinguishes ordinary from supersingular curves in characteristic p .

Goals of this talk

Formalise some reusable results for formal group laws:

- Definitions: formal group laws, commutativity, homomorphisms and isomorphisms.

Goals of this talk

Formalise some reusable results for formal group laws:

- Definitions: formal group laws, commutativity, homomorphisms and isomorphisms.
- Induced algebraic structure: `AddGroup` and `AddCommGroup` on the points of a formal group law.

Goals of this talk

Formalise some reusable results for formal group laws:

- Definitions: formal group laws, commutativity, homomorphisms and isomorphisms.
- Induced algebraic structure: `AddGroup` and `AddCommGroup` on the points of a formal group law.
- Maps on points induced by homomorphisms and isomorphisms.

Formal group laws

Definition

A **one dimensional formal group law** over R is a formal power series $F(X, Y) \in R[[X, Y]]$ such that:

- The constant coefficient is 0.
- The coefficients of X and Y are equal to 1.
- (Associativity) $F(X, F(Y, Z)) = F(F(X, Y), Z)$.

It is called a commutative formal group law if $F(X, Y) = F(Y, X)$.

Remark: Some references include $F(X, 0) = X$ and $F(0, Y) = Y$ in the definition of formal group law. In fact, these two equalities can be deduced from the three conditions above.

Formal group

```
variable (R : Type*) [CommRing R]

name_power_vars X0, X1 over R
name_power_vars Y0, Y1, Y2 over R

@[ext]
structure FormalGroup where
  toPowerSeries : MvPowerSeries (Fin 2) R
  zero_constantCoeff : constantCoeff toPowerSeries = 0
  lin_coeff_X : coeff (single 0 1) toPowerSeries = 1
  lin_coeff_Y : coeff (single 1 1) toPowerSeries = 1
  /- Associativity: `F (F (X, Y), Z) = F (X, F (Y, Z))`. -/
  assoc : toPowerSeries.subst ![toPowerSeries.subst ![Y0, Y1], Y2] =
    toPowerSeries.subst ![Y0, toPowerSeries.subst ![Y1, Y2]] (S := R)

class FormalGroup.IsComm (F : FormalGroup R) : Prop where
  comm : F = F.toPowerSeries.subst ![X1, X0]
```

Algebraic structure: Monoid

```
set_option linter.unusedVariables false in
@[nolint unusedArguments]
def Point (F : FormalGroup R) ( $\sigma$  : Type*) :=
  {f : MvPowerSeries  $\sigma$  R // PowerSeries.HasSubst f}

instance : Add (F.Point  $\sigma$ ) where
  add x y := ⟨F.toPowerSeries.subst ![x.val, y.val], ...⟩

instance : Zero (F.Point  $\sigma$ ) where
  zero := ⟨0, PowerSeries.HasSubst.zero⟩

instance : AddMonoid (F.Point  $\sigma$ ) where
  zero_add x := ...
  add_zero x := ...
  nsmul := nsmulRec
  add_assoc x y z := ...

instance [F.IsComm] : AddCommMonoid (F.Point  $\sigma$ ) where
  add_comm x y := ...
```

Inverse

Proposition

Let $F(X, Y)$ be a formal group law over R . Then there is a unique $i(X) \in R[[X]]$ such that

$$F(X, i(X)) = 0.$$

It serves as an ‘inverse’ of X in the F -sense. Therefore, a formal group law $F(X, Y)$ defines an additive group structure.

Algebraic structure: Group

```
instance : Neg (F.Point  $\sigma$ ) where
  neg f := ⟨F.addInv f.val, ...⟩
```

```
instance : AddGroup (F.Point  $\sigma$ ) where
  zsmul := zsmulRec
  neg_add_cancel x := ...
```

```
instance [F.IsComm] : AddCommGroup (F.Point  $\sigma$ ) where
  add_comm x y := ...
```

Homomorphisms

Definition (Homomorphism)

Let $F(X, Y), G(X, Y)$ be two formal group laws over R . A

homomorphism of formal group laws from $F(X, Y)$ to $G(X, Y)$ is a power series $\alpha(X) = \sum_{i=1}^{\infty} a_i X^i \in R[[X]]$ such that

$$\alpha(F(X, Y)) = G(\alpha(X), \alpha(Y)). \quad (0.1)$$

Homomorphisms

Definition (Homomorphism)

Let $F(X, Y), G(X, Y)$ be two formal group laws over R . A

homomorphism of formal group laws from $F(X, Y)$ to $G(X, Y)$ is a power series $\alpha(X) = \sum_{i=1}^{\infty} a_i X^i \in R[[X]]$ such that

$$\alpha(F(X, Y)) = G(\alpha(X), \alpha(Y)). \quad (0.1)$$

A homomorphism $\alpha(X)$ from $F(X, Y)$ to $G(X, Y)$ is an **isomorphism** if there exists a homomorphism $\beta(X)$ from $G(X, Y)$ to $F(X, Y)$ such that

$$\alpha \circ \beta = \beta \circ \alpha = \text{id}. \quad (0.2)$$

It is called a **strict isomorphism** if $a_1 = 1$.

Homomorphisms: Formalization

```
@[ext]
structure FormalGroupHom where
  toPowerSeries : PowerSeries R
  zero_constantCoeff : toPowerSeries.constantCoeff = 0
  /-- The homomorphism condition:  $\alpha(F(X,Y)) = G(\alpha(X), \alpha(Y))$ . -/
  hom : toPowerSeries.subst F.toPowerSeries = G.toPowerSeries.subst
    (toPowerSeries.toMvPowerSeries .)
```


Homomorphisms: Coercion

```
def FormalGroupHom.applyPoint (f : FormalGroupHom F G) (x : F.Point  $\sigma$ ) :  
  G.Point  $\sigma$  := f.toPowerSeries.subst x.val, ...  
  
instance : CoeFun (FormalGroupHom F G)  
  (fun _  $\mapsto$  { $\sigma$  : Type*}  $\rightarrow$  F.Point  $\sigma \rightarrow$  G.Point  $\sigma$ ) where  
  coe f := f.applyPoint  
  
lemma FormalGroupHom.map_add (f : FormalGroupHom F G) {x y : F.Point  $\sigma$ } :  
  f (x + y) = f x + f y :=  
  
def FormalGroupHom.toAddMonoidHom (f : FormalGroupHom F G) :  
  F.Point  $\sigma \rightarrow$ + G.Point  $\sigma$  where  
  toFun := f.applyPoint  
  map_zero' := ...  
  map_add' _ _ := f.map_add
```

Isomorphisms

```

@[ext]
structure FormalGroupIso where
  toHom : FormalGroupHom F G
  invHom : FormalGroupHom G F
  /-- `toHom ∘ invHom = id`. -/
  left_inv : toHom.toPowerSeries.subst ∘
    (PowerSeries.subst invHom.toPowerSeries) = id
  /-- `invHom ∘ toHom = id`. -/
  right_inv : invHom.toPowerSeries.subst ∘
    (PowerSeries.subst toHom.toPowerSeries) = id

class FormalGroupIso.IsStrict (α : FormalGroupIso F G) : Prop where
  coeff_one : α.toHom.toPowerSeries.coeff 1 = 1

def FormalGroupIso.toEquiv (α : FormalGroupIso F G) :
  F.Point  $\simeq$ + G.Point  $\sigma$  where
  toFun := α.toHom.applyPoint
  invFun := α.invHom.applyPoint
  left_inv x := ...
  right_inv y := ...
  map_add' _ _ := α.toHom.map_add

```

Future work

- **Lubin–Tate theory.** Construction of Lubin–Tate formal group laws, construction of maximal abelian extension of a local field, and establish explicit local class field theory.

Future work

- **Lubin–Tate theory.** Construction of Lubin–Tate formal group laws, construction of maximal abelian extension of a local field, and establish explicit local class field theory.
- **Formal groups of elliptic curves.** Connect with Mathlib’s existing elliptic curve formalization; classify ordinary (height 1) vs. supersingular (height 2) via height.

Future work

- **Lubin–Tate theory.** Construction of Lubin–Tate formal group laws, construction of maximal abelian extension of a local field, and establish explicit local class field theory.
- **Formal groups of elliptic curves.** Connect with Mathlib’s existing elliptic curve formalization; classify ordinary (height 1) vs. supersingular (height 2) via height.
- **Lazard’s theorem.** Identify the universal coefficient ring as $\mathbb{Z}[U_1, U_2, \dots]$.

Future work

- **Lubin–Tate theory.** Construction of Lubin–Tate formal group laws, construction of maximal abelian extension of a local field, and establish explicit local class field theory.
- **Formal groups of elliptic curves.** Connect with Mathlib’s existing elliptic curve formalization; classify ordinary (height 1) vs. supersingular (height 2) via height.
- **Lazard’s theorem.** Identify the universal coefficient ring as $\mathbb{Z}[U_1, U_2, \dots]$.
- **Classification of formal group laws by height.** Over algebraically closed fields of characteristic p , two formal group laws are isomorphic if and only if they have the same height.

Questions

Thanks for your attention!
<https://github.com/WenrongZou/FormalGroupLaws>